

Java 12 — Switch Expressions (Preview)

Java Version: 12

Year: 2019

Feature: Switch Expressions (*Preview*)

UniqueClassName: SwitchExpressions

Definition

Switch expressions were introduced as a **preview feature in Java 12** to make switch more concise and allow it to **produce a value**.

Traditional switch was primarily a statement:

```
switch (day) {  
    case 1:  
        result = "Monday";  
        break;  
    case 2:  
        result = "Tuesday";  
        break;  
}
```

With Java 12's preview feature, switch could be used as an expression:

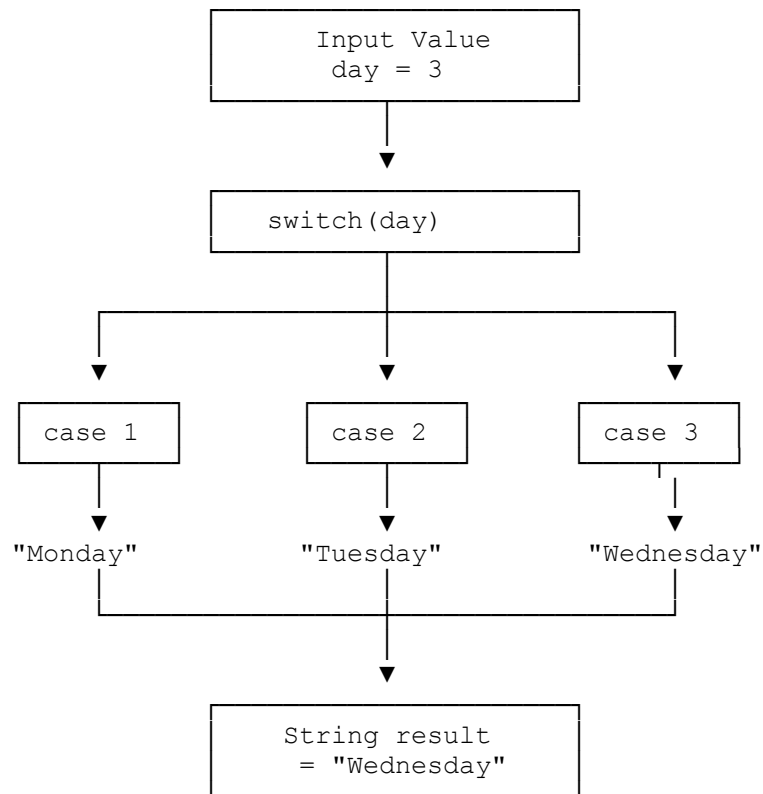
```
String result = switch (day) {  
    case 1 -> "Monday";  
    case 2 -> "Tuesday";  
    default -> "Unknown";  
};
```

Main Benefits

- switch can return a value.
- -> syntax eliminates many break statements.
- Reduces accidental fall-through.
- Makes conditional value assignment more readable.
- Supports multiple case labels in a single branch.

Important: In **Java 12**, switch expressions were a **preview feature**. The syntax evolved during subsequent preview iterations and was finalized in Java 14.

Flowchart



Java Code Example

Because switch expressions were a preview feature in Java 12, compilation requires `--enable-preview`.

SwitchExpressions.java

```
public class SwitchExpressions {  
    public static void main(String[] args) {  
        int day = 3;  
  
        String dayName = switch (day) {  
            case 1 -> "Monday";  
            case 2 -> "Tuesday";  
            case 3 -> "Wednesday";  
            case 4 -> "Thursday";  
            case 5 -> "Friday";  
            case 6 -> "Saturday";  
            case 7 -> "Sunday";  
            default -> "Invalid day";  
        };  
    }  
}
```

```
        System.out.println("Day: " + dayName);
    }
}
```

Output

Day: Wednesday

Multiple Case Labels

Java 12's switch-expression preview also allowed multiple labels to share a result:

```
public class SwitchExpressions {
    public static void main(String[] args) {
        int day = 6;

        String type = switch (day) {
            case 1, 2, 3, 4, 5 -> "Weekday";
            case 6, 7 -> "Weekend";
            default -> "Invalid";
        };

        System.out.println(type);
    }
}
```

Output:

Weekend

This is much shorter than writing separate branches for every value.

Traditional switch vs Switch Expression

Traditional Switch

```
int day = 3;
String dayName;

switch (day) {
    case 1:
        dayName = "Monday";
        break;
```

```

    case 2:
        dayName = "Tuesday";
        break;

    case 3:
        dayName = "Wednesday";
        break;

    default:
        dayName = "Invalid";
}

```

Switch Expression

```

int day = 3;

String dayName = switch (day) {
    case 1 -> "Monday";
    case 2 -> "Tuesday";
    case 3 -> "Wednesday";
    default -> "Invalid";
};

```

The second form directly expresses the relationship:

input → switch → value

Using yield

Java 12's preview version also introduced yield for returning a value from a multi-statement switch branch.

```

public class SwitchExpressions {

    public static void main(String[] args) {

        int marks = 85;

        String grade = switch (marks / 10) {

            case 10:
            case 9:
                yield "A";

            case 8:
                yield "B";

            case 7:
                yield "C";

            default:

```

```

        yield "D";
    };

    System.out.println("Grade: " + grade);
}
}

```

Output:

Grade: A

yield was useful when a case required multiple statements before producing its result.

Compilation in Java 12

Since this was a preview feature, use:

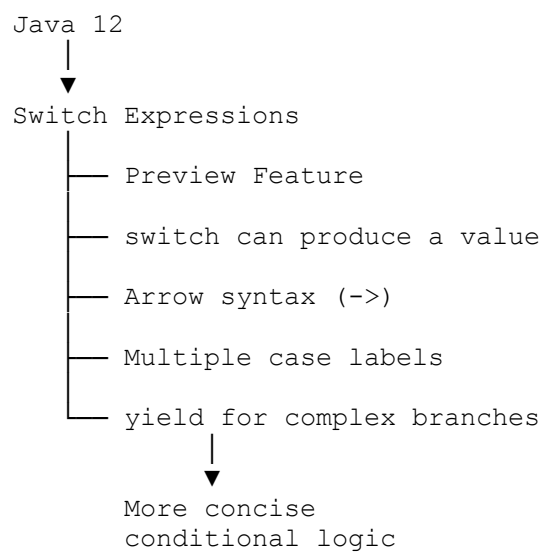
```
javac --enable-preview --release 12 SwitchExpressions.java
```

Run with:

```
java --enable-preview SwitchExpressions
```

The `--enable-preview` option tells Java 12 to enable preview language features.

Key Point



In short: Java 12 introduced **switch expressions as a preview feature**, allowing switch to return a value and providing the more concise case ... -> ... syntax. The feature was subsequently refined and became a permanent part of Java in **Java 14**.